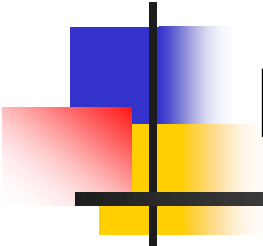
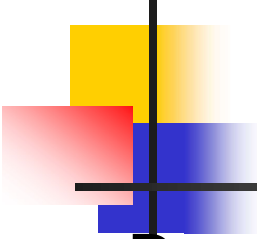


Algoritmos e Estrutura de Dados



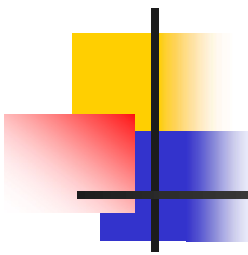
Professor: Geraldo Pereira de Souza



Qualidade de código

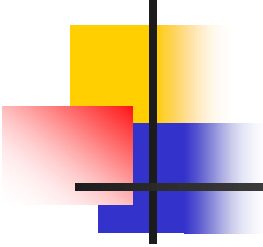
Para determinarmos se um programa é bom, diversos fatores devem ser analisados. Entre eles, alguns dos mais importantes são:

- Do ponto de vista do usuário: se o programa está correto, se é eficiente e se é de fácil utilização;
- Do ponto de vista da equipe de programadores: se é fácil fazer alterações, seja para correção de erros ou para atualizações.



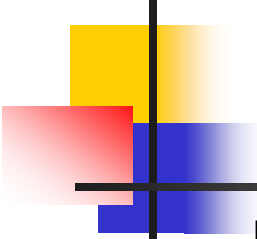
Como Medir Eficiência de um Algoritmo?

- Um programa é eficiente quando faz bom uso dos recursos computacionais existentes.
- Recursos importantes:
 - Memória
 - Tempo de CPU



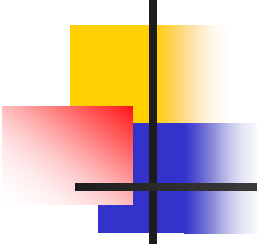
Eficiência de Tempo?

- Quando economizamos o tempo de CPU para realizar determinado processamento, dizemos que temos eficiência de tempo;
- Quando economizamos memória disponível, dizemos que temos eficiência de espaço.



Curiosidade sobre desempenho

- Dentro de um programa P temos 2 métodos (ou funções):
 - a() leva 0,0001 segundo na execução
 - b() leva 0,002 segundo na execução
- Os dois métodos são usados dentro do programa P um número X e Y de vezes respectivamente. Quais considerações você deve fazer sobre desempenho global em casos de baixa performance?



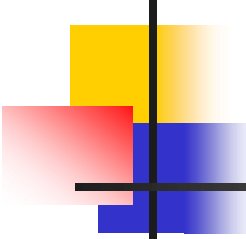
Considerações para AED

- Quanto `a questão da utilização dos programas do ponto de vista do usuário, deixaremos a discussão para a disciplina Interface Homem-Máquina
- Para AED, estamos interessados nos fatores de qualidade do ponto de vista do programador, visto que estes garantirão que o programa seja alterado com mais facilidade quando for necessário.



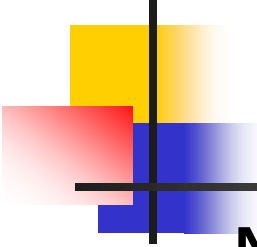
Características

- Os elementos que chamam a atenção quando falamos em qualidade do código são:
 - a identificação,
 - os comentários,
 - a documentação,
 - a escolha dos identificadores e
 - modularização.



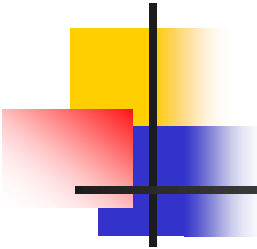
Características

- A identificação permite visualizar as relações de dependência entre as diversas construções do programa.
- Os comentários ajudam a entender algoritmos complexos e a resolver dúvidas que possam surgir ao ler um fragmento de código.



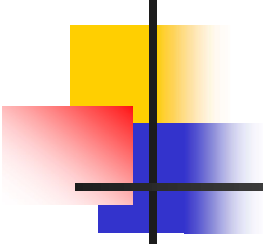
Importante: Identificadores

- Nome de variáveis significativos:
 - Exemplo: matriculaAluno, salario, idade, etc;
- Idem para nome de métodos:
 - Exemplo: CalcularCpmf(), InserirAluno(), ExcluirAluno(), etc;
- Casos especiais:
 - x para expressão matemática,
 - i, j para contadores de loops;



Importante: Modularização

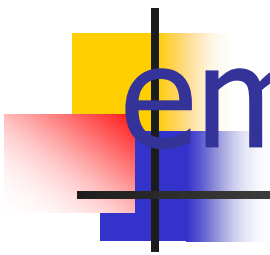
- A modularização é a divisão do programa em funções, classes e módulos;
- Ao dividir o programa em partes, facilitamos sua leitura, visto que é possível desenvolver métodos menores independentes;



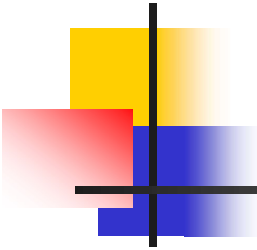
Importante: Tamanho de método

- Corpo do método deve caber em uma página do editor para facilitar a visualização, leitura e entendimento! Sem usar barras de rolagem.

Importante: O que modularizar em um programa?

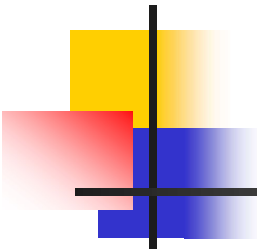


- Sempre que possível agrupe funcionalidades em métodos!
- As partes redundantes de um programa devem ser substituídas por chamadas de funções!



Exemplo de Modularização

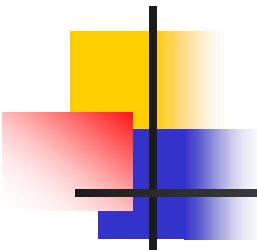
```
void main(void) {  
    int vetor1[10], vetor2[10], hamming;  
    leVetor(vetor1);  
    leVetor(vetor2);  
    hamming = calculaDistanciaHamming(vetor1, vetor2);  
    cout << "Distancia de hamming = " << hamming << endl;  
}
```



Programação Defensiva???

No trânsito existe o dito popular: “Você deve dirigir para você e para os outros!”

Na programação, o ditado pode ser adaptado para:
“Programar para você e para os outros!”

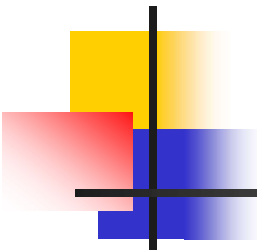


Programação Defensiva???

Qual seria o resultado se y for igual a ZERO?

```
int divide(int x, int y) {  
    return (x/y);  
}
```

- Uma boa regra de programação a ser seguida: teste todos os valores que a função for utilizar, sejam parâmetros, entradas do usuário, ou variáveis globais.



Programação Defensiva???

Estruturas como try / catch / finally torna possível um tratamento mais robusto e elegante. Será visto mais adiante.

```
int divide(int x, int y) {  
    return (x/y);  
}
```


Dependência entre funções:

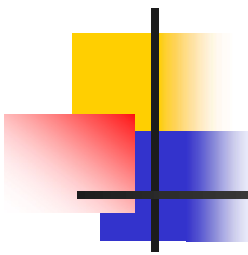
Retorno de funções

- Retorno das funções: se uma função F1 chama uma função F2 e utiliza o resultado retornado por esta, então estaremos estabelecendo uma relação de dependência.
- Muitas vezes, esta relação pode ser confusa. Veja exemplo a seguir que retorna a posição de um elemento ou -1, caso o elemento não seja encontrado!

Dependência entre funções:

Retorno de funções

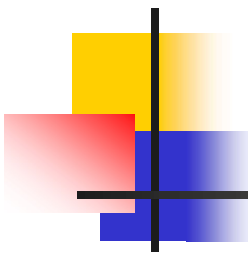
```
int posicaDe(int v[], int n, int x) {  
    bool encontrado = false;  
    int i = 0;  
    while (i < n && encontrado == false) {  
        if (v[i] == x) encontrado = true;  
        else i++;  
    }  
    if (encontrado == true) return i;  
    else return -1;  
}
```



Dependência entre funções: Retorno de funções

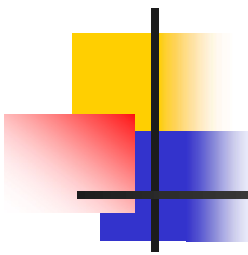
- O retorno está sendo usado para duas funcionalidades distintas!
- Uma solução melhor é o uso de exceções que torna o código mais robusto e será visto mais adiante!

Dependência entre funções: Parâmetros



- Se uma função F1 chama uma função F2, então é importante que ao escrever a função F1, estejamos cientes dos parâmetros que F2 espera.

Dependência entre funções: Parâmetros

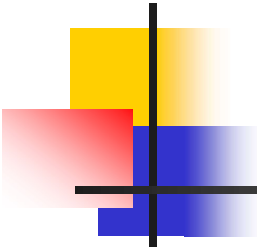


- Qual declaração abaixo você escolheria?
 - void extenso(int dia, int mes, int ano);
 - void extenso(int ano, int mes, int dia);
 - void extenso(Data d);

Problemas de interpretação dependendo do país!

Dependência entre funções: Variáveis Globais

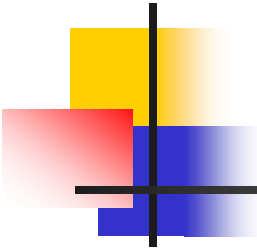
```
int x; // Variável global  
F2(...) {  
    ...  
    x = ...; // variável x alterada  
    ...  
}  
F1(...) {  
    ...  
    F2(...)  
    ... x ... // uso do valor de x  
}
```



Outras Considerações

O que a OO ajuda?

O que o uso de bibliotecas ajuda? Exemplo: bibliotecas para tratar datas, campos numéricos, etc;



Referência

- Apostila de AED dos professores Fábio Tirelo e Silvio Jamil, disponível no SGA.
- Livros da ementa de AED disponibilizados no plano de ensino.